

Tema 3: Introducción a la biblioteca SDL

Tecnología de la Programación de Videojuegos 1

Grado en Desarrollo de Videojuegos

Curso 2026-2027

Miguel Gómez-Zamalloa Gil con de Rubén Rubio Cuéllar
cambios

Departamento de Sistemas Informáticos y Computación

Universidad Complutense de Madrid

¿Por qué SDL?

- ✦ Cada plataforma tiene su propia forma de manejar la salida gráfica, la entrada del usuario, acceder al hardware, etc.
- ✦ SDL proporciona **acceso uniforme y eficiente** a todas estas características específicas de cada plataforma (audio, teclado, ratón, joystick, gráficos 2D y 3D vía OpenGL o Vulkan, etc.)
 - ▶ ¡Más tiempo para centrarse en el desarrollo propio del juego!
- ✦ **Multiplataforma:** Windows, macOS, Linux, iOS, Android y consolas
- ✦ Activamente mantenido y distribuido → manuales, tutoriales, foros, etc.
 - ▶ Ver wiki.libsdl.org

Nuevo en SDL 3

- ♦ Callbacks en lugar de bucle de juego (opcional)
- ♦ Soporte GPU para gráficos y cómputo
- ♦ Cuadros de diálogo del sistema
- ♦ Cámaras y lápices
- ♦ Procesos externos
- ♦ Entrada/salida asíncrona
- ♦ Cambios en la API (nombres de funciones, etc.)

Nuevo en SDL 2

- ♦ Aceleración 2D (hardware) y 3D
- ♦ Soporte para múltiples ventanas
- ♦ Soporte para entrada de tipo multi-touch
- ♦ Múltiples fuentes de audio y múltiples fuentes de entrada
- ♦ Captura de audio
- ♦ Mejoras en programación multi-hebra
- ♦ Manejo del portapapeles, rueda horizontal del ratón, etc.

Inicialización y finalización

```
#include <SDL3/SDL.h>
constexpr int WIN_WIDTH = 800;
constexpr int WIN_HEIGHT = 600;

int main(int argc, char* argv[]) {
    // Variables
    SDL_Window* window = nullptr;
    SDL_Renderer* renderer = nullptr;
    // Inicialización del sistema, ventana y renderer
    SDL_Init(SDL_INIT_VIDEO);
    window = SDL_CreateWindow("First test with SDL", WIN_WIDTH, WIN_HEIGHT, 0);
    renderer = SDL_CreateRenderer(window, nullptr);
    if (window == nullptr || renderer == nullptr) // En general lanzaremos una excepción
        cout << "Error initializing SDL: " << SDL_GetError() << "\n";
    else { // Programa que usa SDL
        ...
    }
    // Finalización
}
```

Inicialización y finalización

```
#include <SDL3/SDL.h>
constexpr int WIN_WIDTH = 800;
constexpr int WIN_HEIGHT = 600;

int main(int argc, char* argv[]) {
    // Variables
    ...
    // Inicialización del sistema, ventana y renderer
    ...
    if (window == nullptr || renderer == nullptr) // En general lanzaremos excepción
        cout << "Error initializing SDL: " << SDL_GetError() << "\n";
    else { // Programa que usa SDL
        ...
    }
    // Finalización
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
}
```

Dibujando con SDL

```
#include <SDL3/SDL.h>
constexpr int WIN_WIDTH = 800;
constexpr int WIN_HEIGHT = 600;

int main(int argc, char* argv[]) { // Tiene que ser así exactamente
    // [...] Variables
    // [...] Inicialización del sistema, ventana y renderer
    if (window == nullptr || renderer == nullptr)
        cout << "Error initializing SDL\n";
    else { // Programa que usa SDL
        SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255); // RGB y alpha
        SDL_RenderClear(renderer); // Borra la pantalla
        SDL_RenderPresent(renderer); // Muestra la escena
        SDL_Delay(5000); // Espera 5 segs. antes de cerrar
    }
    // Finalización
    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
}
```

Dibujando con SDL - Texturas

SDL maneja dos estructuras básicas para hacer dibujos, `SDL_Surface` y `SDL_Texture`:

- ✦ `SDL_Surface` es una colección de píxeles almacenados en la CPU con funciones para leer, escribir y manipular su contenido
- ✦ `SDL_Texture` permite el renderizado acelerado por hardware (GPU). ¡Es la que usaremos!

Creación de texturas:

```
SDL_Texture* texture; // Variable para la textura
string filename = "..."; // Nombre del fichero con la imagen .bmp
SDL_Surface* surface = SDL_LoadBMP(filename.c_str()); // 0 LoadPNG
texture = SDL_CreateTextureFromSurface(renderer, surface);
SDL_DestroySurface(surface); // Se borra la estructura auxiliar
// Textura lista para ser usada
```

Texturas - ciclo de vida

Típicamente las texturas se crean, se utilizan repetidamente en el juego (repintados) y al final del programa se destruyen:

1. Creación: (ver diapositiva anterior)
2. Ejemplo de ciclo de repintados: sea `vector<SDL_Texture*> textures`

```
while (!exit){  
    // Actualizar posiciones de objetos del juego  
    SDL_RenderClear(renderer); // Borra la pantalla  
    for (int i = 0; i < textures.size(); ++i)  
        SDL_RenderTexture(renderer, textures[i], ...); // Pinta la textura  
    SDL_RenderPresent(renderer); // Muestra la escena  
}
```

3. Destrucción:

```
for (SDL_Texture* texture : textures)  
    SDL_DestroyTexture(texture); // Borra memoria dinámica
```

Rectángulos fuente y destino

- ♦ El **rectángulo fuente** define el área que queremos copiar de la textura a la ventana SDL
- ♦ El **rectángulo destino** define el área de la ventana SDL en la que queremos copiar el rectángulo fuente
- ♦ Sean srcRect y destRect del tipo **SDL_FRect** los rectángulos fuente y destino, se usan así:

```
SDL_RenderTexture(renderer, textures[i], &srcRect, &destRect);
```

- ♦ Obsérvese que se esperan punteros a **SDL_FRect** (por eso el &)
- ♦ Si se pasa **nullptr** en un rectángulo se interpreta como el rectángulo fuente/destino que ocupa toda la textura/ventana

Rectángulos fuente y destino

El tipo `SDL_FRect` es una estructura con cuatro campos:

```
struct SDL_FRect {  
    float x;    // Coordenada x de la esquina superior izqda.  
    float y;    // Coordenada y de la esquina superior izqda.  
    float w;    // Anchura del rectángulo  
    float h;    // Altura del rectángulo  
}
```

Ejemplo 1: Pintamos toda la textura pero escalada a un frame de 50x50

```
SDL_FRect destRect;  
destRect.w = destRect.h = 50; // Frame de 50x50  
destRect.x = destRect.y = 0; // Se pinta en la esquina superior izqda  
SDL_RenderTexture(renderer, texture, nullptr, &destRect);  
SDL_RenderPresent(renderer); // Muestra la escena
```

Rectángulos fuente y destino

Ejemplo 2: Imaginamos una matriz virtual de 8x8 en la escena y pintamos el primer frame (fantasma rojo) escalado a la celda (0,1) y el frame 11º (primer pacman) a la celda (4,3):

```
float textW, textH; // Para saber el tamaño de la textura
SDL_GetTextureSize(texture, &textW, &textH);
SDL_FRect srcRect, destRect;
srcRect.w = textW/14; srcRect.h = textH/4; // Tamaño frame textura
uint cellW = WIN_WIDTH/8; uint cellH = WIN_HEIGHT/8;
destRect.w = cellW; destRect.h = cellH; // Tamaño celda salida
srcRect.x = srcRect.y = 0; // Frame fantasma rojo
destRect.x = 1*cellW; destRect.y = 0*cellH; // Celda (0,1)
SDL_RenderTexture(renderer, texture, &srcRect, &destRect);
srcRect.x = 10 * textW/14; // Frame pacman
destRect.x = 3*cellW; destRect.y = 4*cellH; // Celda (4,3)
SDL_RenderTexture(renderer, texture, &srcRect, &destRect);
SDL_RenderPresent(renderer); // Muestra la escena
```



Rectángulos fuente y destino

Ejemplo 3: Creando una animación a partir de un "sprite-sheet"



Cada `TIME_PER_FRAME` milisegundos movemos el rectángulo fuente una posición a la derecha

```
int textFrameW = textW / 6;
int textFrameH = textH / 1;
SDL_FRect srcRect;
srcRect.x = srcRect.y = 0;
srcRect.w = textFrameW;
srcRect.h = textFrameH;
while (!exit) {
    srcRect.x = textFrameW * ((SDL_GetTicks() / TIME_PER_FRAME) % 6);
    SDL_RenderClear(renderer);
    SDL_RenderTexture(renderer, texture, &srcRect, nullptr);
    SDL_RenderPresent(renderer); // Muestra la escena
}
```

El paquete SDL_Image

- ✦ SDL por defecto solo nos permite usar ficheros **bmp** y **png**
- ✦ El paquete **SDL_Image** permite la carga de otros formatos de ficheros de imágenes (**gif**, **jpeg**, **tiff**, **avif**, etc.)
- ✦ Una vez instalado e incluido mediante

```
#include <SDL3_image/SDL_image.h>
```

- ✦ Simplemente hay que sustituir el

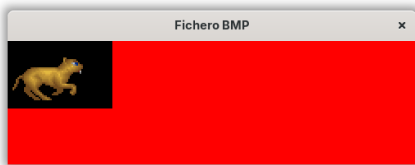
```
SDL_Surface* surface = SDL_LoadBMP(filename.c_str());
```

y su conversión a textura por la llamada equivalente

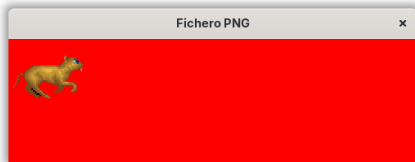
```
SDL_Texture* texture = IMG_LoadTexture(renderer, filename.c_str());
```

Ficheros png

- ♦ Los ficheros **png** a parte de tener un tamaño muy pequeño tienen soporte para transparencia (**canal alfa** del color)
- ♦ Esto permite fundir imágenes con el fondo. Por ejemplo, nos permitiría en lugar de ver la imagen así:



verla de esta forma:



Manejo básico de eventos

- ♦ SDL mantiene una cola con los eventos pendientes de ser procesados
- ♦ La función `bool SDL_PollEvent(SDL_Event* event)` devuelve si hay algún evento pendiente y en tal caso el primero de ellos en la variable `event`
- ♦ El tipo `SDL_Event` es una unión (`union`) de estructuras (`struct`) diversas para cada tipo de evento. El atributo común `type` indica el tipo de evento y a los atributos particulares de cada tipo se accede con `event.key.key`, `event.key.repeat`, etc. Ver documentación de SDL para los distintos tipos de eventos
- ♦ Algunos ejemplos de tipos de eventos `SDL_EVENT_X` donde `X` puede ser `WINDOW_MOVED`, `KEY_DOWN`, `KEY_UP`, `MOUSE_BUTTON_DOWN`, `MOUSE_MOTION`, `FINGER_DOWN`, `PEN_MOTION`, `SENSOR_UPDATE`, ...

Manejo básico de eventos

Ejemplo: bucle que trata todos los eventos pendientes y procesa algunos de ellos

```
SDL_Event event;
...
while (!exit) {
    ...
    while (SDL_PollEvent(&event) && !exit) {
        switch (event.type) {
            case SDL_EVENT_QUIT: exit = true; break;
            case SDL_EVENT_KEY_DOWN:
                if (event.key.key == SDLK_DOWN) ...
                else if (event.key.key == SDLK_UP) ...
            case SDL_EVENT_MOUSE_BUTTON_UP:
                if (event.button.button == SDL_BUTTON_LEFT) ...
        }
    }
    ...
}
```

Manejo alternativo de la entrada

En algunos casos es preferible consultar el estado de los dispositivos de entrada en lugar de reaccionar a los eventos.

- ✦ `SDL_GetKeyboardState` devuelve el estado del teclado

```
const bool *keys = SDL_GetKeyboardState(nullptr);  
if (keys[SDL_SCANCODE_SPACE]) { /* ... */ }
```

- ✦ `SDL_GetMouseState` sirve para consultar el estado del ratón

```
float x, y;  
SDL_MouseButtonFlags flags = SDL_GetMouseState(&x, &y);  
if (flags & SDL_BUTTON_MASK(SDL_BUTTON_LEFT)) { /* ... */ }
```

- ✦ Otras funciones para consultar el estado de joysticks, etc.

Aún así, es necesario consumir los eventos para que el estado se actualice.

Control básico del tiempo

- ♦ La función `Uint64 SDL_GetTicks()` nos devuelve el tiempo actual en milisegundos
- ♦ La función `void SDL_Delay(Uint32 delay)` suspende el programa `SDL delay` milisegundos
- ♦ Hay versiones con el sufijo `NS` que devuelven/reciben nanosegundos
- ♦ Tenemos dos alternativas básicas para controlar que el juego solo actualice cada `FRAME_PERIOD` milisegundos:
 1. Contar el tiempo consumido en una iteración y si no se ha llegado a `FRAME_PERIOD` suspender el programa el tiempo correspondiente
 2. Contar el tiempo consumido y si no se ha llegado a `FRAME_PERIOD` repintar y tratar eventos, pero NO actualizar

Control básico del tiempo

Alternativa 1 (suspendiendo el programa):

```
uint64_t startTime, frameTime;

while (!exit) { // Bucle del juego
    startTime = SDL_GetTicks();
    handleEvents();
    update(); // Actualiza el estado de todos los objetos del juego
    render(); // Renderiza todos los objetos del juego
    frameTime = SDL_GetTicks() - startTime; // Tiempo de la iteración
    if (frameTime < FRAME_PERIOD) // Suspende por el tiempo restante
        SDL_Delay(FRAME_PERIOD - frameTime);
}
```

Es más eficiente (suelta el procesador y ahorra energía) pero dependiendo del juego puede resultar inaceptable.

Control básico del tiempo

Alternativa 2 (actualizar solo cada FRAME_PERIOD):

```
uint64_t startTime, frameTime;
startTime = SDL_GetTicks();

while (!exit) { // Bucle del juego
    handleEvents();
    // Tiempo desde última actualización
    frameTime = SDL_GetTicks() - startTime;
    if (frameTime >= FRAME_PERIOD) {
        update(); // Actualiza el estado de todos los objetos del juego
        startTime = SDL_GetTicks();
    }
    render(); // Renderiza todos los objetos del juego
}
```

Mayor control y precisión aunque consume más recursos

Control básico del tiempo

Alternativa 1bis (en función del tiempo transcurrido, *delta time*):

```
uint64_t startTime, frameTime;

while (!exit) { // Bucle del juego
    startTime = SDL_GetTicks();
    handleEvents();
    update(FRAME_PERIOD); // <-- recibe el tiempo transcurrido
    render();
    frameTime = SDL_GetTicks() - startTime;
    if (frameTime < FRAME_PERIOD)
        SDL_Delay(FRAME_PERIOD - frameTime);
}
```

Se puede cambiar FRAME_PERIOD sin cambiar la velocidad del juego